CSE 202

Algorithm Design and Analysis
Lecture 5:, Dijkstra's Algorithm

Graph reachability:

procedure GraphSearch (G: directed graph, s: vertex)

```
Initialize X = empty, F = {s}, U = V – F.
While F is not empty:
  Pick w in F.
  For each neighbor y of v:
      If y is not in X or F:
          move u from U to F.
  Move w from F to X.

Return X.
```

Graph reachability:

procedure GraphSearch (G: directed graph, s: vertex)

Initialize X = empty, F = {s}, U = V – F.
While F is not empty:
  Pick w in F.
  For each neighbor y of v:
    If y is not in X or F:
      move u from U to F.
  Move w from F to X.

Return X.

How do we pick w?
How do we list w's neighbors?
How do we move elements from one set to another?

Graph reachability:

procedure GraphSearch (G: directed graph, s: vertex)

Initialize X = empty, F = {s}, U = V – F.
While F is not empty:
  Pick w in F.
  For each neighbor y of v:
    If y is not in X or F:
      move u from U to F.
  Move w from F to X.

Return X.

Data structures:
G: adjacency list
Array indexed by vertex, three values (X,F,U)
A stack or queue to store F

Graph reachability (low level)

procedure GraphSearch (G: directed graph, s: vertex)

```
Initialize Status[1...|V|] to all U; Status[s]=F;
                PushF(s)
While F.head ≠ NIL do:
   w=PopF;
  For each  y in N(w):
     IF
        Status[y]==U THEN Status[y]=F; PushF(y)
    For v in V : IF Status[v]=X then output v.
```

Data structures:
G: adjacency list
Array indexed by vertex, three values (X,F,U)
A stack or queue to store F

Time analysis attempt 1

procedure GraphSearch (G: directed graph, s: vertex)

Initialize Status[1...|V|] to all U; Status[s]=F;
PushF(s) O(|V|)
While F.head $\neq NIL\ do$: ???
w=PopF; O(1)+ deg(w) = O(|V|)
For each y in N(w):
IF Status[y]==U **THEN** Status[y]=F; PushF(y)
For v in V : **IF** Status[v]=X **then** output v. O(|V|)

Number of iterations
Before we push y, Status[y]=U
After, Status[y]=F
Never goes back to U
So we only push each vertex once.
So we only pop each vertex once.

Total time: O(|V|*number of iterations)= $O(|V|^2)$

Time analysis attempt 2

procedure GraphSearch (G: directed graph, s: vertex)

Initialize Status[1...|V|] to all U; Status[s]=F;
PushF(s) O(|V|)
While F.head $\neq NIL\ do$: ???
w=PopF; O(1)+ deg(w) = O(|V|)
For each y in N(w):
IF Status[y]==U **THEN** Status[y]=F;
PushF(y)
For v in V : **IF** Status[v]=X **then** output v.
O(|V|)

Total time: O(|V|+|E|)

We only pop each vertex once. Therefore, total time in while loop is at most

$$\sum_{w \in V} O(1 + \deg(w))$$

=

$$\sum_{w \in V} O(1) + \sum_{w \in V} O(\deg(w))$$

=O(|V|+|E|)

DOES WHICH DATA STRUCTURE WE USE MATTER?

Both stacks and queues give O(|V|+|E|) algorithms.

What's the difference?

Queue=

Stack =

DOES WHICH DATA STRUCTURE WE USE MATTER?

Both stacks and queues give O(|V|+|E|) algorithms.

What's the difference?

Queue= Breadth-first search

Stack = Depth-first search
Priority queue= Dijkstra's algorithm.

DIFFERENCES

F is stack: Last-found is first explored. Depth-first search

F is queue: First found is first explored. Breadth-first search

F is priority queue based on total length: Dijkstra's algorithm

STACK VERSUS QUEUE

The queue gives us some extra information.

The queue starts with just the node s, the only one that has distance 0.

Then you put each neighbor of s into the queue and now the queue has all the nodes at distance 1 from s

Then you put each neighbor of those into the queue one after the other until you have all the nodes at distance 2 from s.

In fact, for each distance d from s, the queue contains all the nodes at distance d from s and nothing else. As these are ejected, their undiscovered neighbors are the next nodes added to the queue.

BREADTH FIRST SEARCH (BFS)

Given a graph G and a starting vertex s,

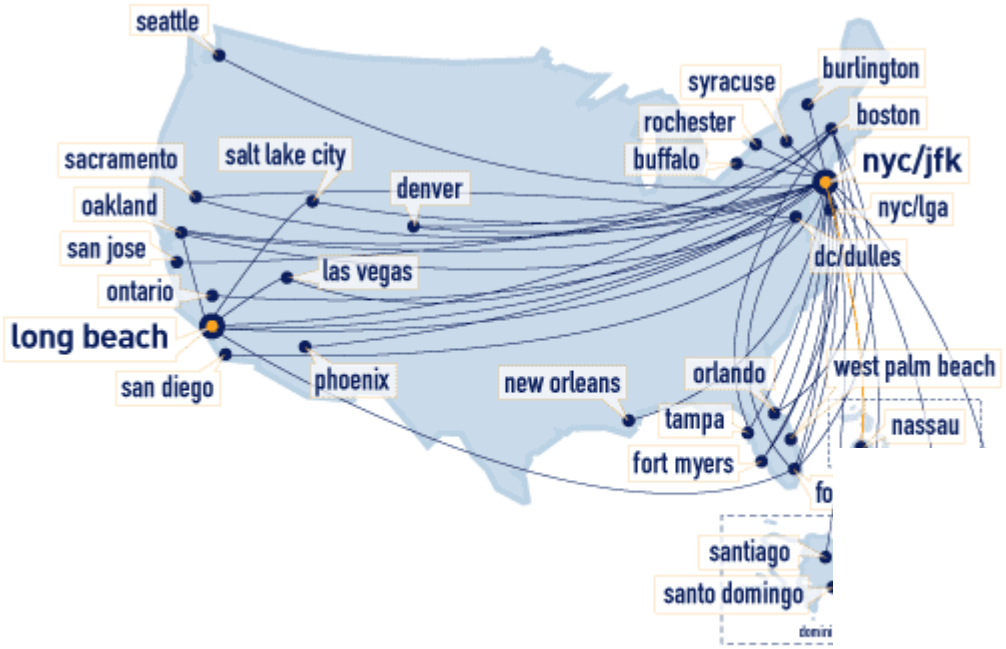
BFS computes distances from s to every other node. It keeps this information in an array dist.

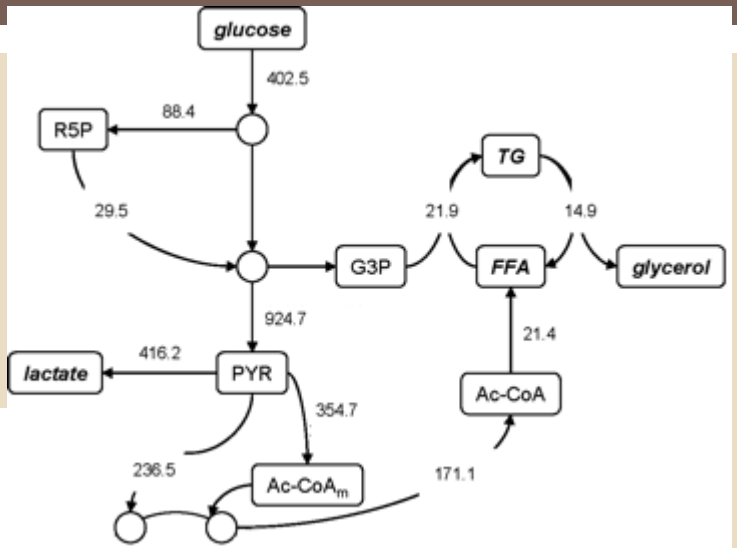
To do this, BFS computes distances “layer by layer.”

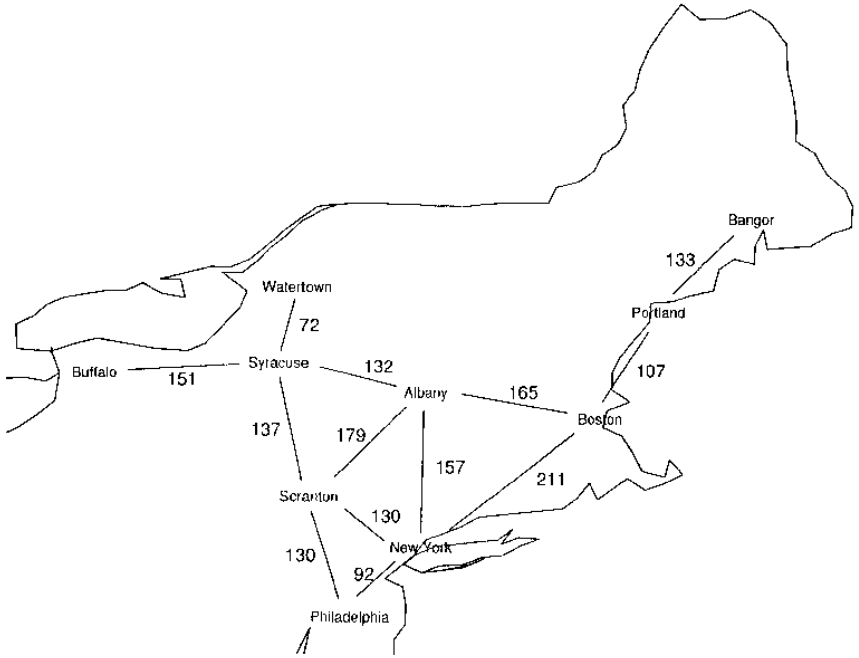
It sets dist(s)=0 and the next layer is all the vertices adjacent to s. If v is adjacent to s then it sets dist(v)=1.

For each vertex v, BFS will set dist(v) = dist(prev(v))+1

GRAPHS WITH EDGE LENGTHS

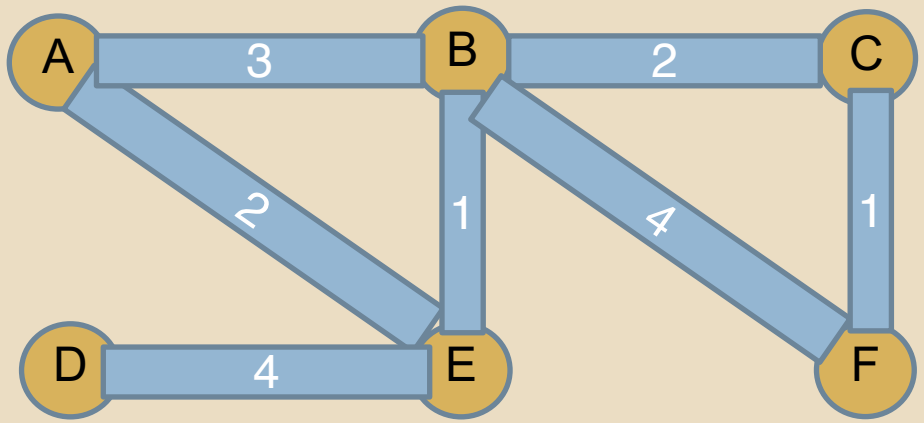






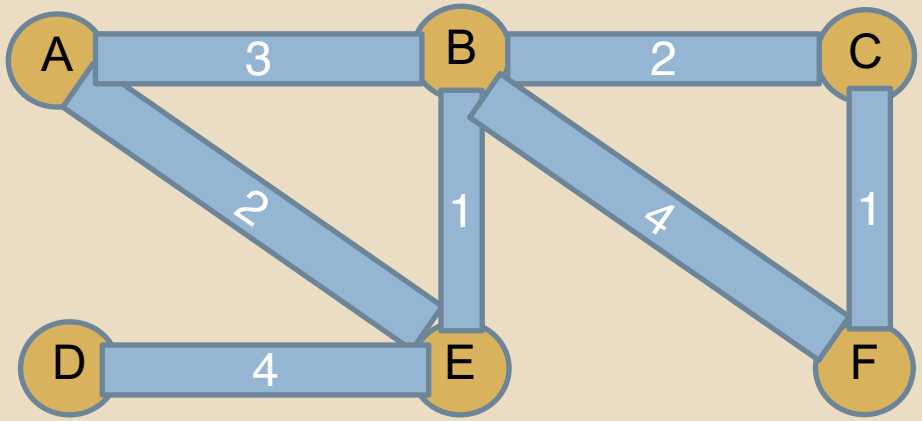
BFS ON WEIGHTED GRAPHS.

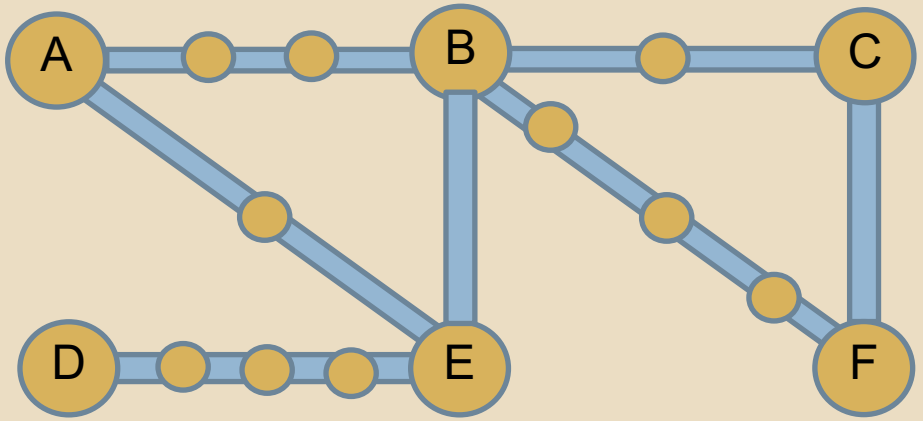
BFS only works to find shortest distances on graphs in which each edge has equal weight.



BFS ON WEIGHTED GRAPHS.

- On a graph G with integer edge lengths, form G' by adding $\ell_e - 1$ many new vertices between u and v for every edge $e = (u, v)$. Then run BFS on G'.

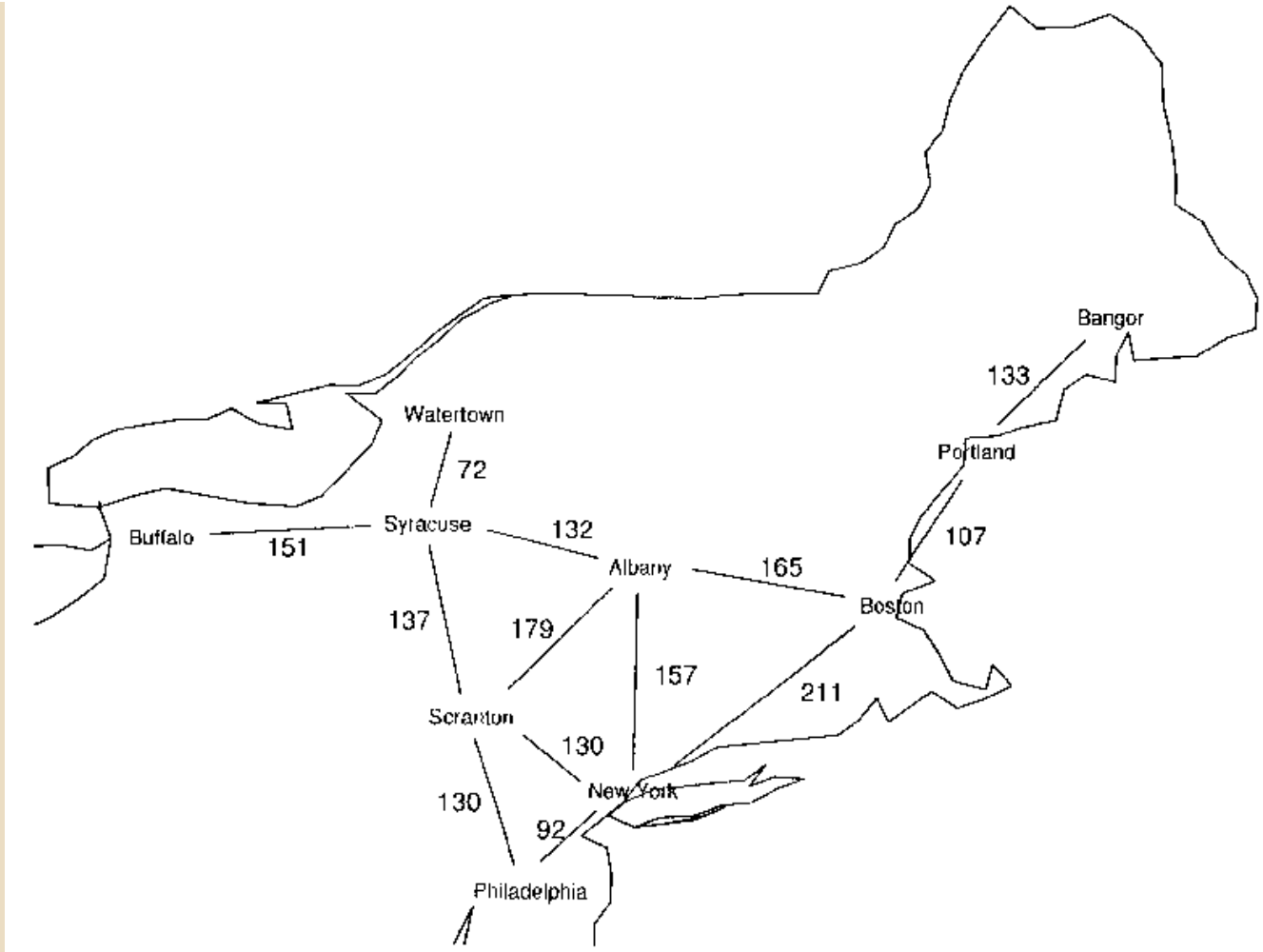




PROBLEMS WITH THIS METHOD

If the edge lengths (weights) are large integers then it is impractical.

In this example with 10 vertices, we must add 1,783 more vertices!!!!!!!!



HIGH LEVEL TO LOW LEVEL ALGORITHM DESIGN

While our end goal is to develop fully specified algorithms, with all of the details including data structures, and a complete time analysis, it is useful to first come up with a **high level** version of the algorithm first

A high-level version specifies **what** the algorithm wants to do at every step, in terms of abstractions such as sets, relations, orderings, graphs, and so on, but not all of the details of **how** it will do these steps.

We then need to fill in those details. Usually we make a **low-level algorithm** from a high-level one by matching abstract mathematical structures to **data structures**

ADVANTAGES OF HIGH-LEVEL DESIGN

Clarity: presenting a high-level algorithm first gives the reader (and you) the main idea of what's going on, before getting caught up in details

Correctness proofs are usually much easier to do for high-level algorithms. Then we just need to make sure that the low-level version does what the high-level version specifies

Flexibility. By showing that the high-level algorithm works, we show that **any** low-level implementation will solve the problem. Then we can change the details of the implementation to fit a given situation (e.g., dense vs. sparse graphs, memory efficiency vs. time efficiency, parallel vs. sequential) without worries.

SINGLE SOURCE SHORTEST PATHS PROBLEM

Instance: a graph G = (V,E) (directed).

with non-negative edge lengths $\ell(e) \geq 0$ for each e in E

s: the starting vertex

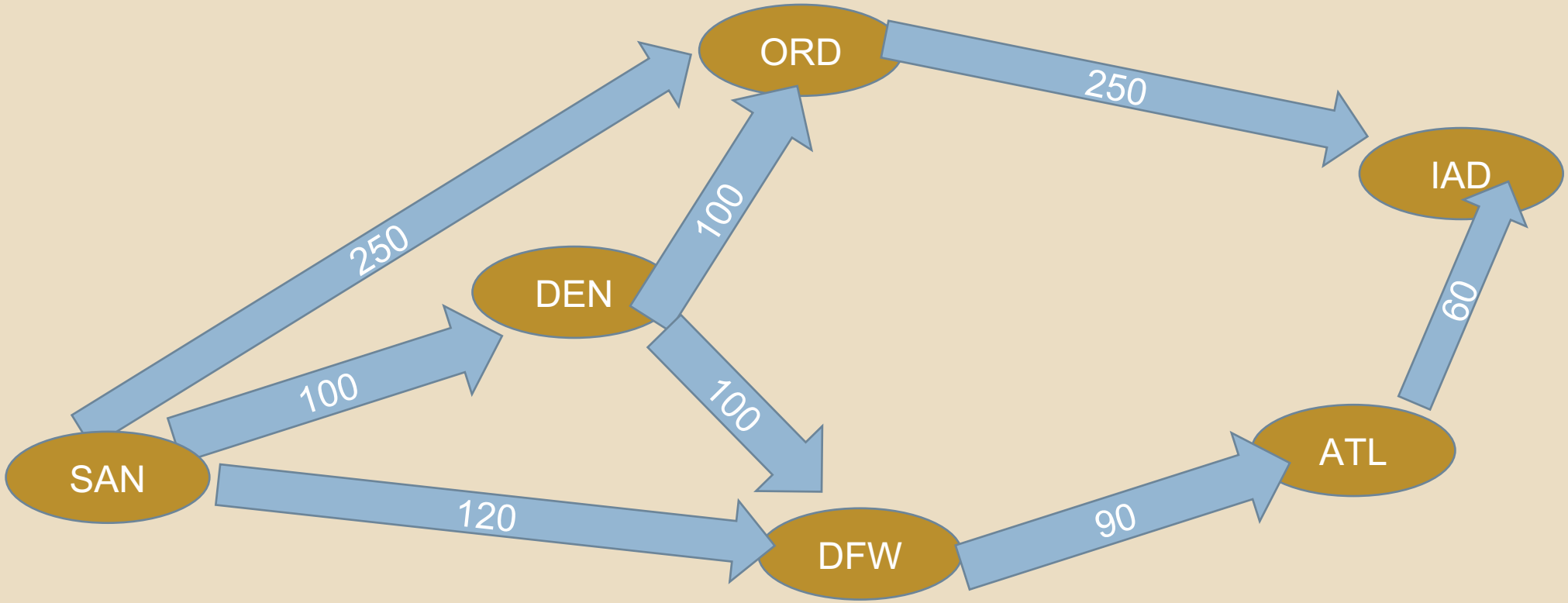
Output: for all vertices u reachable from s, dist(u) is set to

the distance from s to u, i.e., the minimum over all paths from s to u of the sum of the edge lengths.

For all vertices u unreachable from s,

dist(u) is set to infinity.

DIJKSTRA EXAMPLE: WHICH VERTEX WOULD WE REACH FIRST IF WE USED RED TO BFS?



DIJKSTRA'S ALGORITHM, HIGH LEVEL

procedure dijkstra(G,ℓ,s)
X: set of vertices we know distances to
F: set of vertices we aren't sure of the distance to, includes ones we haven't reached

Initialize X to be empty, F = V. Initialize dist(s): =0, dist(u) := infinity for other u.
Initialize prev(u) to be nil for all u

Until F is empty:
Find the vertex $u \in F$ with smallest dist(u).
For each edge $(u, v) \in E$ do:
dist(v)= min(dist(v), dist(u)+l((u,v))) (Note: this step is called "relaxing'')
If dist(v) changed, set prev(v)=u
Move u from F to X (Note: this step is called ``ejecting'' u)
Return the arrays dist(u), prev(u)

``BEST FIRST'' SEARCH

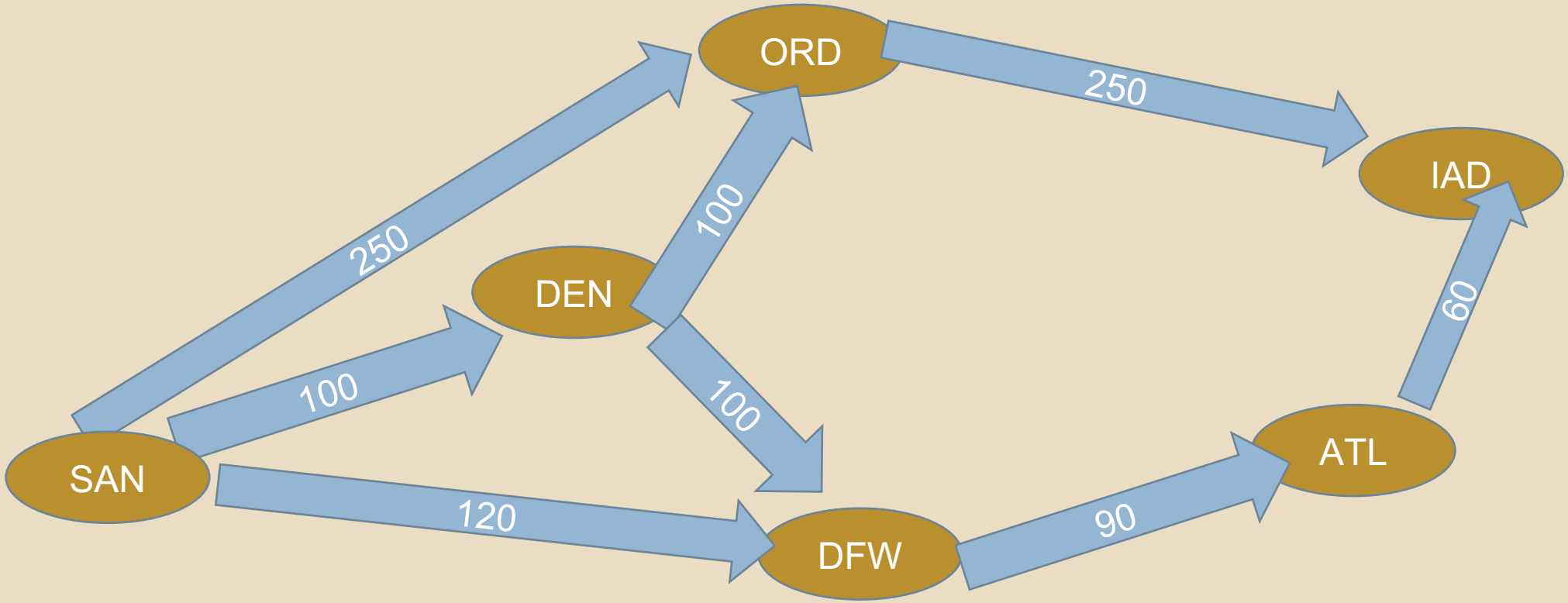
There is tension between two constraints on search:

Correctness: Need to re-explore vertices if we find a new or better path

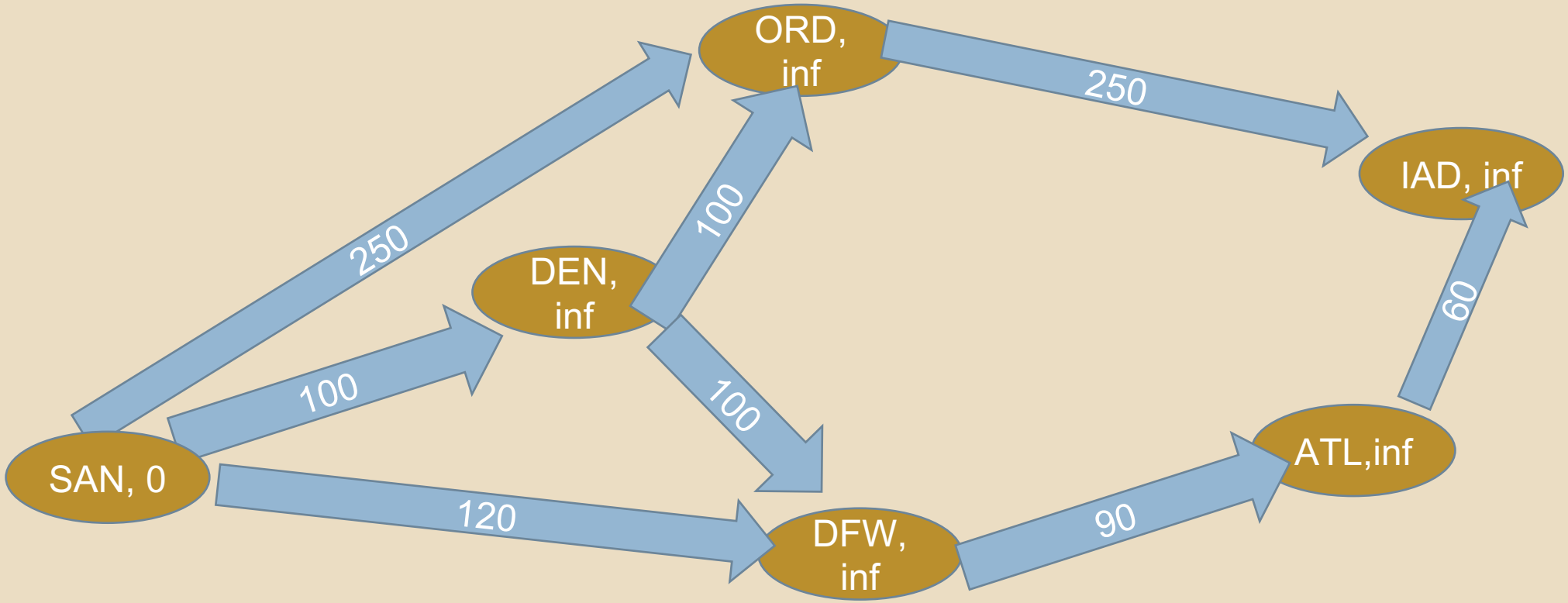
Efficiency: Fast time requires us to explore vertices a very small number of times.

Dijkstra applies a Best First strategy: By exploring vertices in order of their distance, we guarantee that the FIRST path we find to v is the BEST path to v.

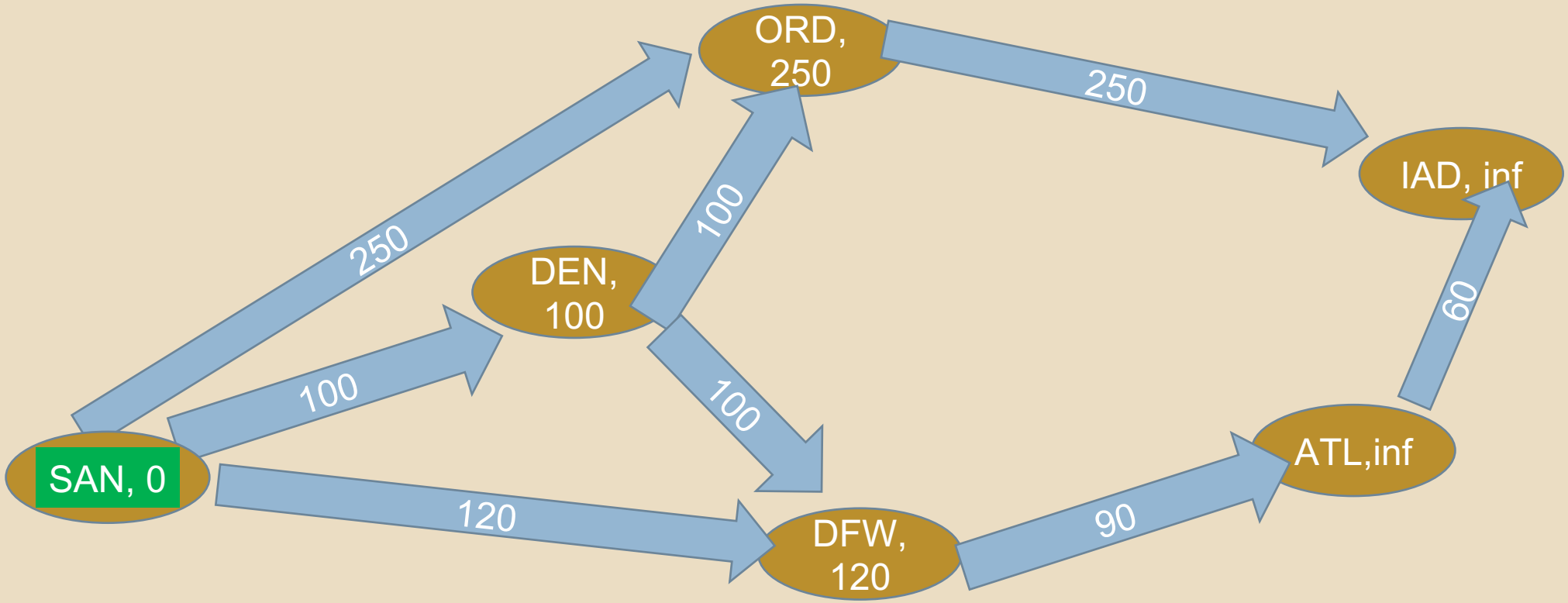
DIJKSTRA EXAMPLE



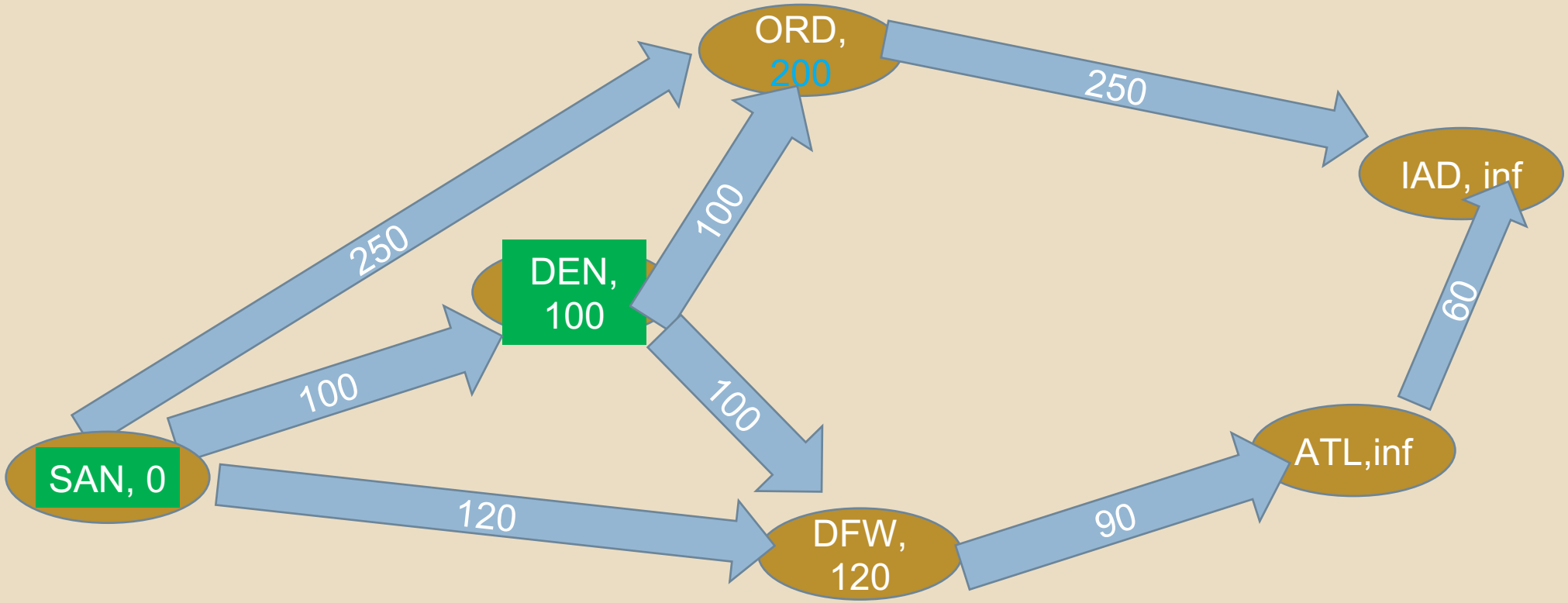
DIJKSTRA EXAMPLE



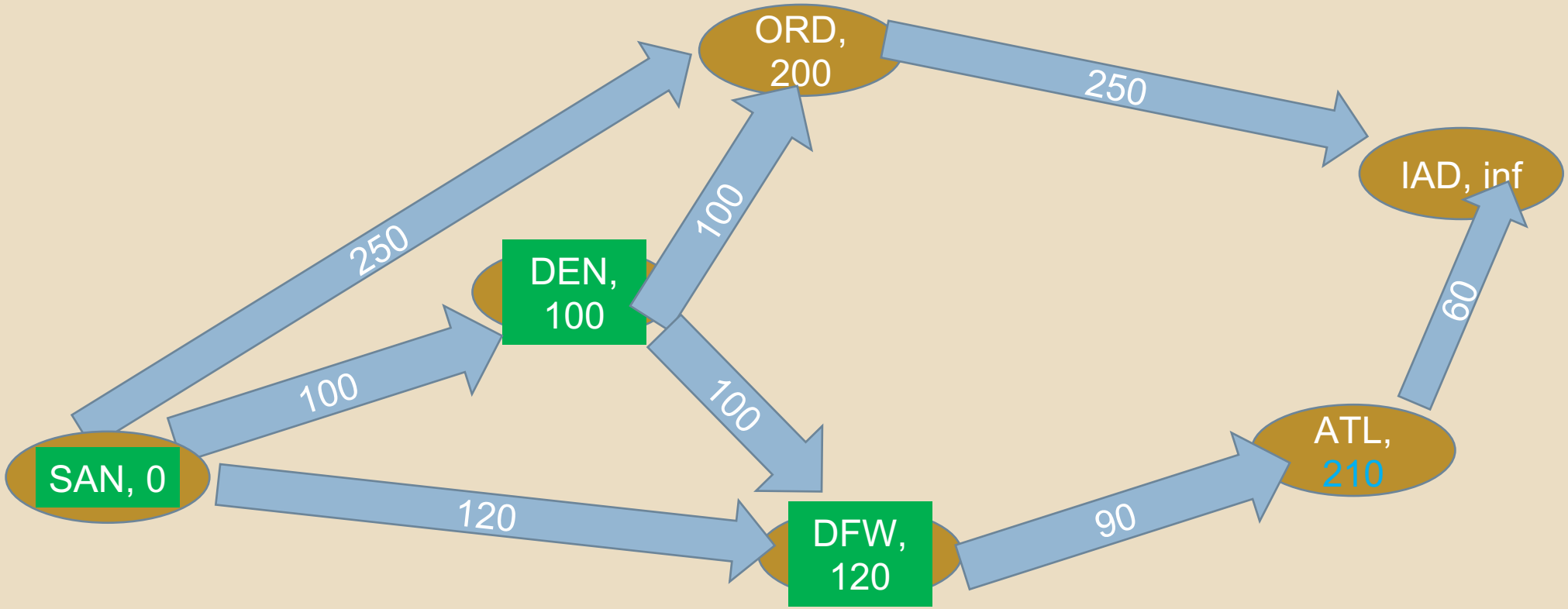
DIJKSTRA EXAMPLE



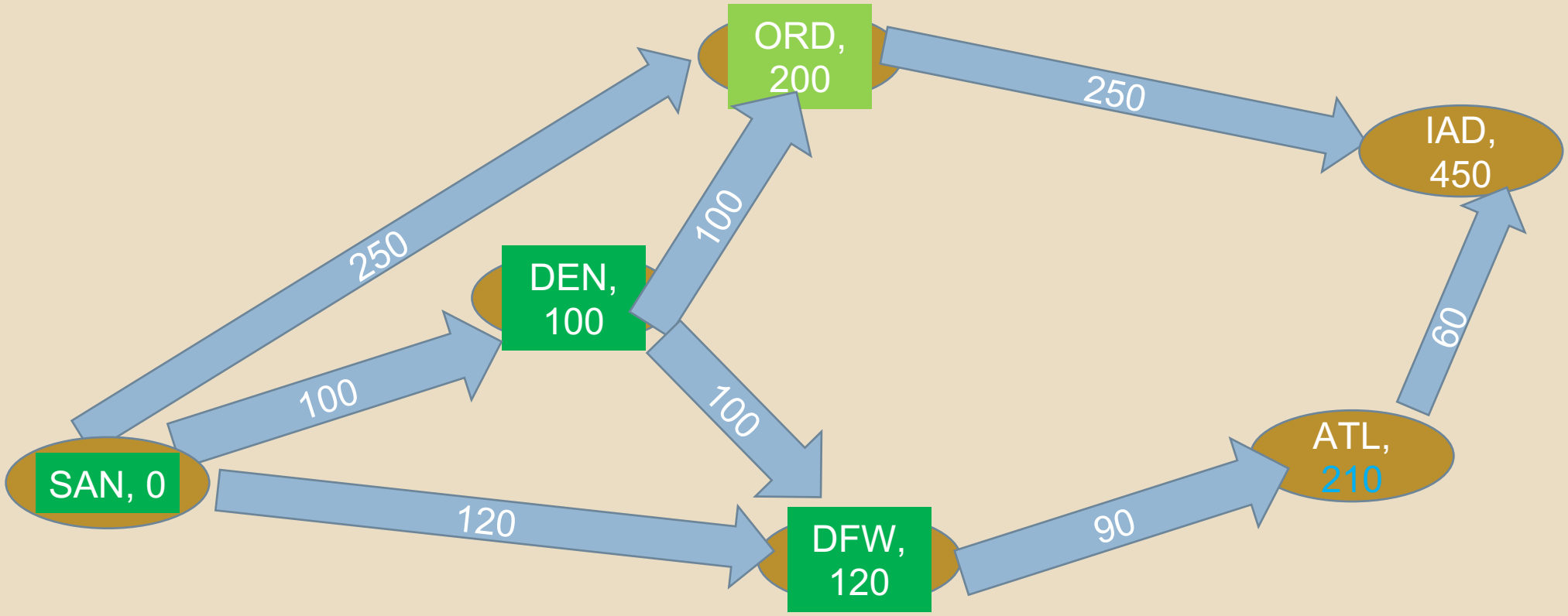
DIJKSTRA EXAMPLE



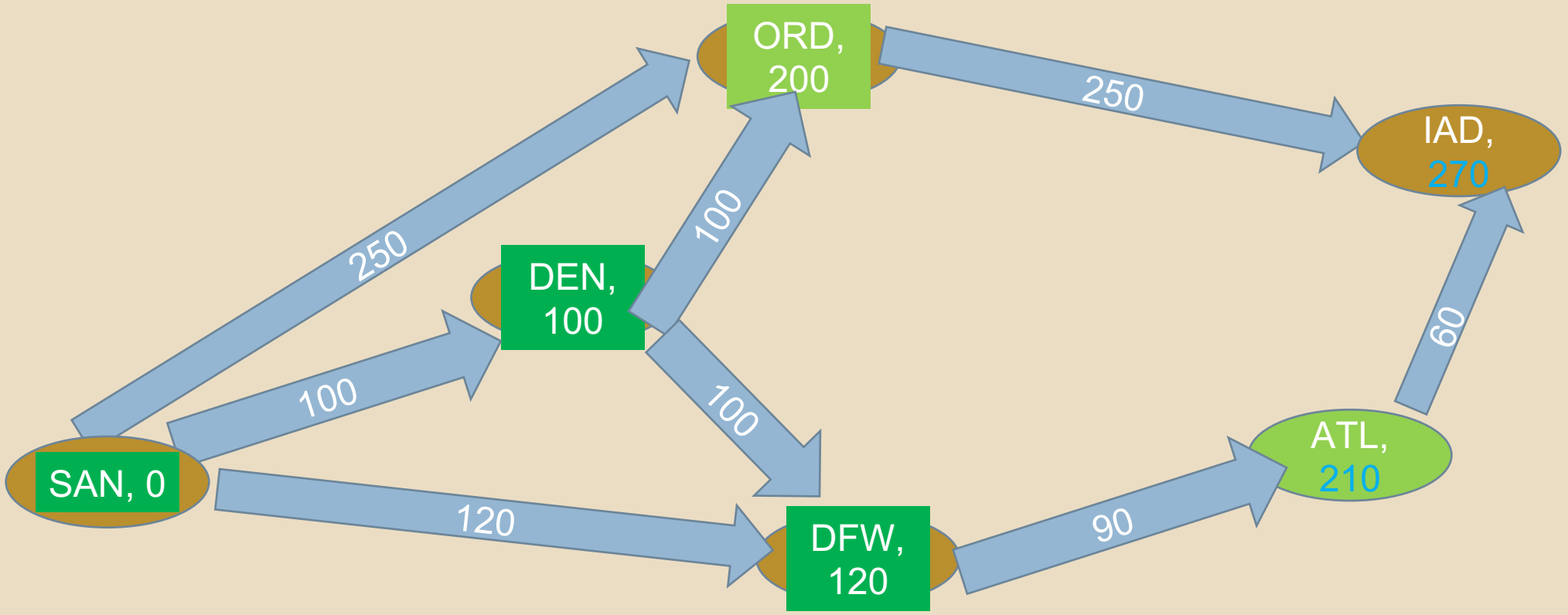
DIJKSTRA EXAMPLE



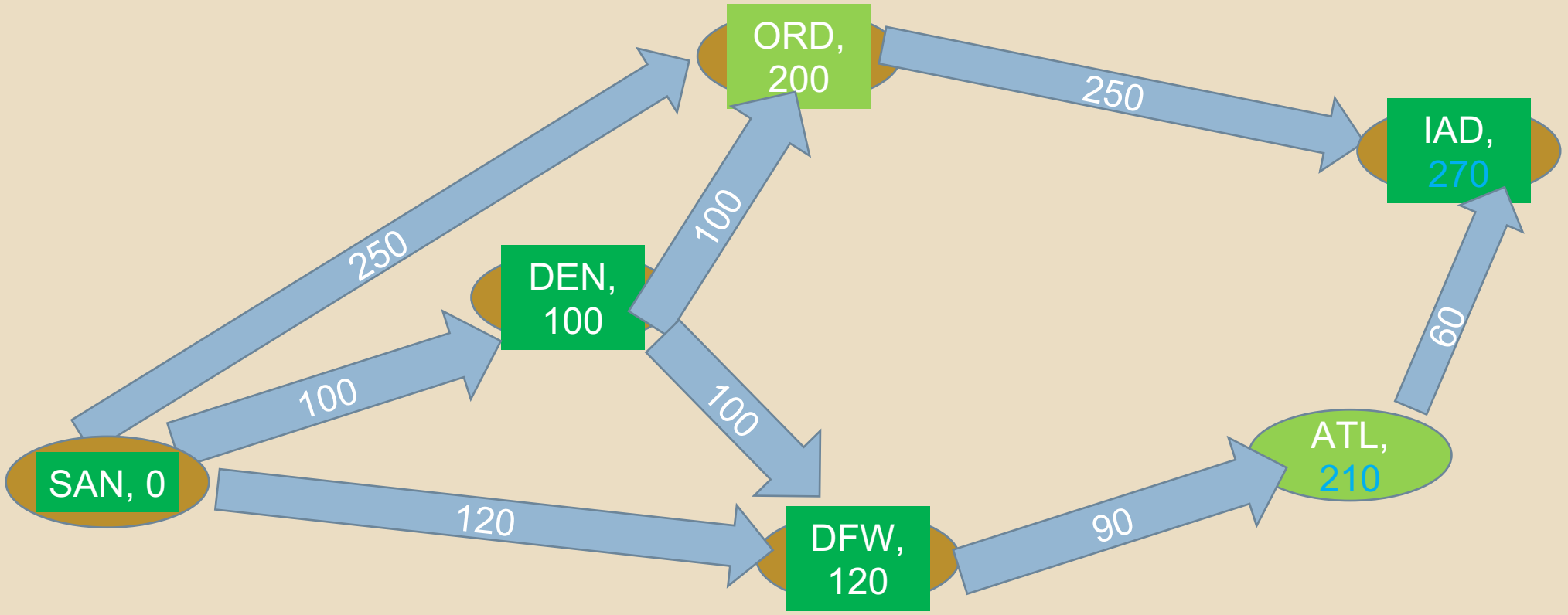
DIJKSTRA EXAMPLE



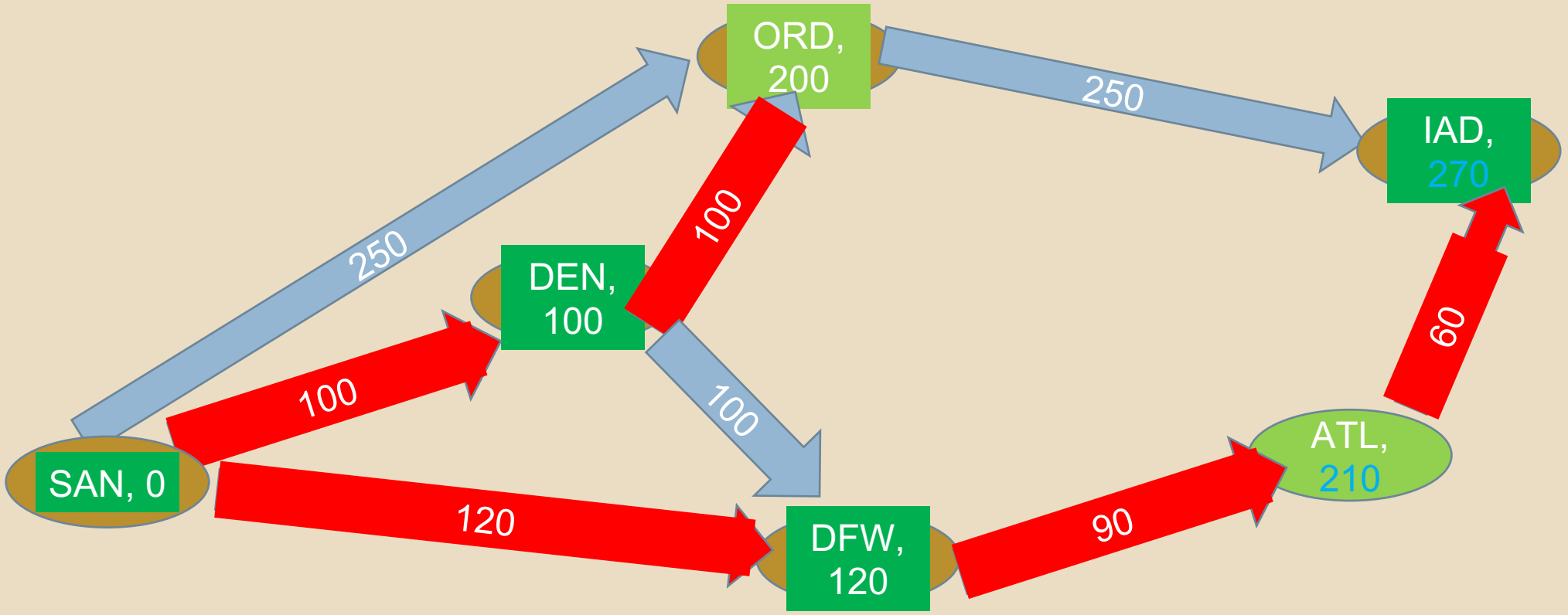
DIJKSTRA EXAMPLE



DIJKSTRA EXAMPLE



DIJKSTRA EXAMPLE



USEFUL INVARIANTS

Lemma: after any number of iterations, for every $u \in F$,

dist(v) = $\min_{u \in X, (u,v) \in E} dist(u) + l((u,v))$ (where, if no such u exists, we view the minimum as infinity)

Proof: After the first iteration, the only possible u is u=s, and the formula is true for N(s), and infinite for other v.

When we add something to X, we update the minimum distances above.

ANOTHER USEFUL INVARIANTS

Lemma: After any number of iterations, for all

$$u \in X, v \in F, dist(u) \leq dist(v)$$

After the first iteration, the only u is s, and dist(v) is either infinity or the length of the edge from s to v.

In the i'th iteration, the vertex u_i we chose was a vertex in F of smallest distance. By the invariant, it now has the largest distance in X. Since it had the smallest distance in F, all other vertices in F had at least as large a distance. If we relax a distance for v, we relax it to $dist(u_i) + l\big((u_i, v)\big) \geq dist(u_i)$, so it is still larger than any distance in X.

COROLLARY

Lemma: After any number of iterations, for all

$$u \in X, v \in F, dist(u) \leq dist(v)$$

Corollary: For the list of vertices in order of ejection, $u_1 = s, u_2, .. u_n$,

$$dist(u_1) \leq dis(u_2) \leq dist(u_3) \ldots \leq dist(u_n)$$

Proof: In the i'th iteration, the vertex u_i we chose was a vertex in F. By the lemma, it now has the largest distance in X.

CORRECTNESS OF DIJKSTRA'S

Let truedist(v) be the actual shortest distance from s to v.

We want to show that at the end of the algorithm dist(v) = truedist(v) for all vertices v.

In the middle of the algorithm, this is not necessarily true for all vertices. But once a vertex moves to X, its dist is locked in.

We will prove that dist(v) = truedist(v) for all v in X.

We will prove this for $v = u_i$ by strong induction on i

CORRECTNESS OF DIJKSTRA'S

- Base case: the starting vertex is put into X. X={s}, dist(s) = 0. Since trudist(s) = 0, this is correct.
- Inductive Hypothesis: For some k≥ 1, assume that when |X|=k, for all vertices v in X, dist(v) = truedist(v).
- (WTS: that after the next ejection (k+1), that vertex is assigned its correct distance value. Since no other distance value is changed.)

CORRECTNESS OF DIJKSTRA'S

Let truedist(v) be the actual shortest distance from s to v.

We want to show that at the end of the algorithm dist(v) = truedist(v) for all vertices v.

We will prove that dist(v) = truedist(v) when v=u_i $is\ moved\ into\ X$, by strong induction on i

We know $dist(u_1) = truedist(u_1), , \ldots dist(u_{i-1}) = truedist(u_{i-1})$

We know $dist(u_i) = min_j\ dist(u_j) + l((u_j, u_i))$ **over j<i with** $(u_j, u_i) \in E$

We know $dist(u_i) \leq dist(u')$ **for all** $u' \in F$

CORRECTNESS OF DIJKSTRA'S

We will prove that dist(v) = truedist(v) when v=u_i $is\ moved\ into\ X$, by strong induction on i

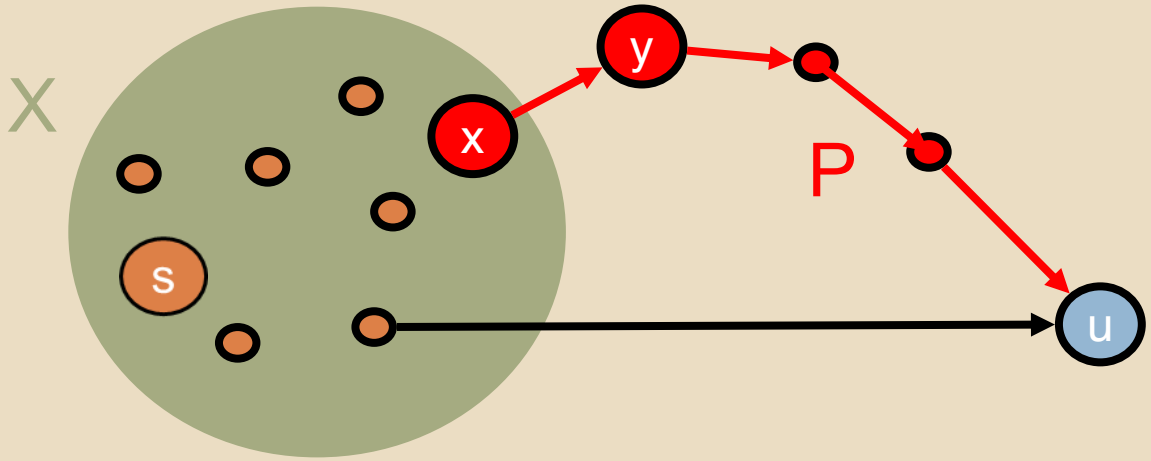
We know $dist(u_1) = truedist(u_1), , \ldots dist(u_{i-1}) = truedist(u_{i-1})$

We know $dist(u_i) = min_j\ dist(u_j) + l((u_j, u_i))$ **over j<i with** $(u_j, u_i) \in E$

So $dist(u_i) = dist(u_j) + l\left((u_j, u_i)\right) = truedist(u_j) + l\left((u_j, u_i)\right) \geq truedist(u_i)$, **so if we are wrong, we are too large.**

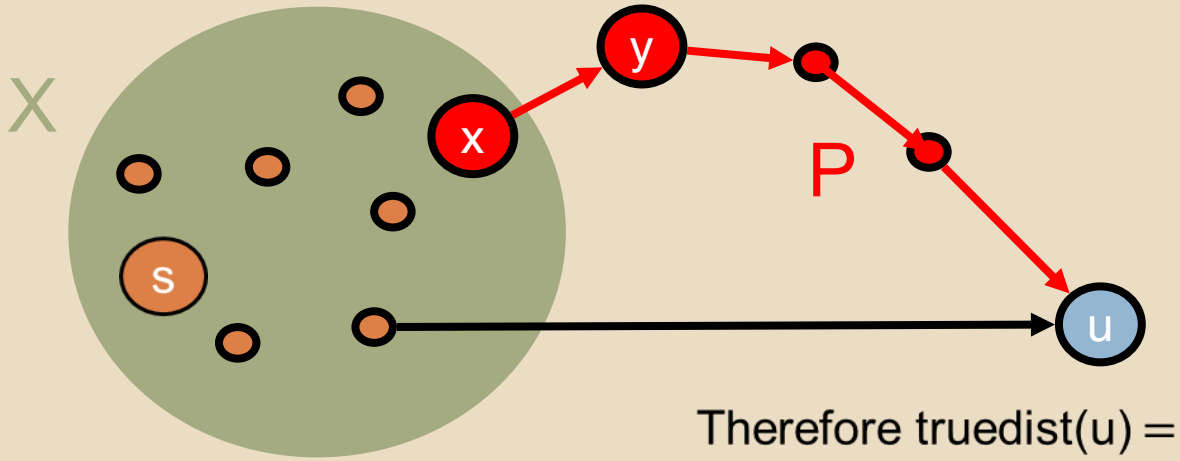
CORRECTNESS OF DIJKSTRA'S

- Inductive step: after k ejections, let u be the next vertex to be ejected. With v its neighbor in X: dist(v)+ℓ(v,u) is the minimum of all edges leaving X.
  - Let's call the true shortest path P.



CORRECTNESS OF DIJKSTRA'S

- Inductive step: after k ejections, let u be the next vertex to be ejected. With v its neighbor in X: dist(v)+ℓ(v,u) is the minimum of all edges leaving X.
  - Let's call the true shortest path P.



There must be some edge (x,y) in P with x$\in X$ and $y \notin X$.

dist(x) + ℓ(x,y) $\geq$ dist(v) + ℓ(v,u) = dist(u) (by choice of u)

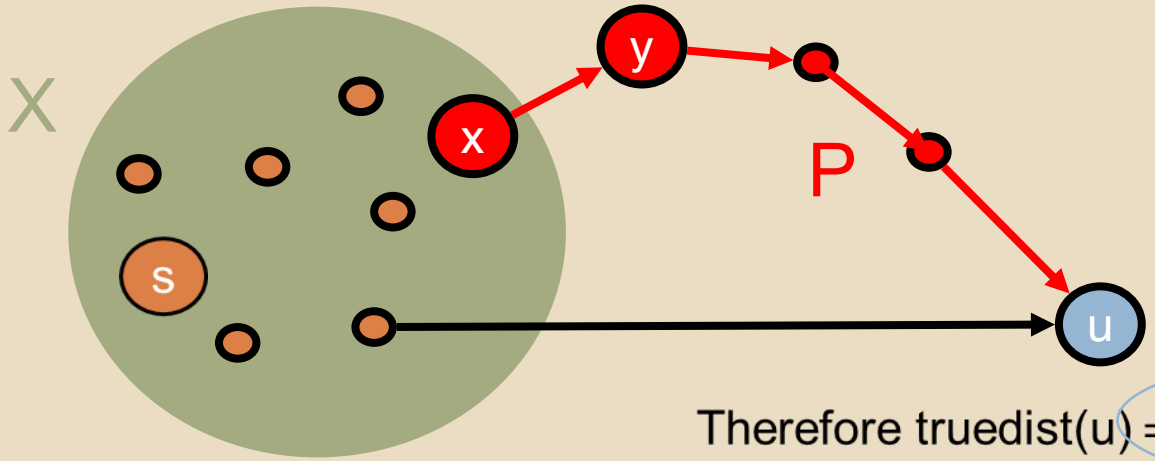
dist(x) = truedist(x) (by IH)

Therefore truedist(u) = len(P) $\geq$ truedist(x) + $\ell(x,y) \geq$ dist(u) > truedist(u)

Contradiction.

WHERE DID WE USE NON-NEGATIVITY?

WHERE WE USE NON-NEGATIVITY



There must be some edge (x,y) in P with x∈ X and $y \notin X$.

dist(x) + ℓ(x,y) ≥ dist(v) + ℓ(v,u) = dist(u) (by choice of u)

dist(x) = truedist(x) (by IH)

Therefore truedist(u) = len(P) ≥ truedist(x) + $\ell(x, y)$ ≥ dist(u) > truedist(u)

Contradiction.

DATA STRUCTURES FOR DIJKSTRA'S ALGORITHM

procedure dijkstra(G,ℓ,s)

Initialize X to be empty, F = V. Initialize dist(s): =0, dist(u) := infinity for other u.
Initialize prev(u) to be nil for all u

Until F is empty:
Find the vertex $u \in F$ with smallest dist(u).
For each edge $(u, v) \in E$ do:
dist(v)= min(dist(v), dist(u)+l((u,v))) (Note: this step is called "relaxing'')
If dist(v) changed, set prev(v)=u
Move u from F to X (Note: this step is called ``ejecting'' u)
Return the arrays dist(u), prev(u)

STRUCTURES IN DIJKSTRA

Graph G: How does it change? Where do we access it?

Set X: How does it change? Where do we access it?

Set F : How does it change? Where do we access it?

STRUCTURES IN DIJKSTRA

Graph G: No changes, need to list members: Adjacency list

Set X: Insert, check membership: array of booleans

Set F : Find and delete the element with minimum key, dist(u).
Decrease the keys of some elements u’

PRIORITY QUEUE

A priority queue is a data structure of a set of objects (vertices) along with key values for each object that can be changed (alarm settings.) And it can support the following operations.

- insert(H,u)
- deletemin(H)
- decreasekey(H,u)
- makequeue(S)

DIJKSTRA'S IN TERMS OF PRIORITY QUEUE

```
procedure dijkstra(G,ℓ,s)
for all u in V
        dist(u) := infinity
        prev(u) := nil
dist(s) := 0
H := makequeue(V)
while H is not empty
        u := deletemin(H)
        for all edges (u,v) in E
                if dist(v) > dist(u) + ℓ(u,v) then
                        dist(v):= dist(u) + ℓ(u,v)
                        prev(v):=u
                        decreasekey(H,v)
```

makequeue(V)
deletemin(H)
decreasekey(H,v)

RUNTIME OF DIJKSTRA'S

```
procedure dijkstra(G,ℓ,s)
for all u in V
        dist(u) := infinity
        prev(u) := nil
dist(s) := 0
H := makequeue(V)
while H is not empty
        u := deletemin(H)
        for all edges (u,v) in E
                if dist(v) > dist(u) + ℓ(u,v) then
                        dist(v):= dist(u) + ℓ(u,v)
                        prev(v):=u
                        decreasekey(H,v)
```

deletemin × |V|
decreasekey × |E|

LOW-LEVEL DIJKSTRA

We've reduced the problem of creating a low-level efficient version of Dijkstra's algorithm to that of finding a good implementation of a priority queue

Total time will be O(n * Time to delete min+ m * time to decrease key)

Different implementations of priority queue will have different trade-offs between costs of operations. Might not be a single one that is optimal for all kinds of graphs. So flexibility is useful

ARRAY AS A PRIORITY QUEUE

Array: indexed by vertex, giving key value.

[key(A),key(B),key(C),key(D),key(E),key(F)]

makequeue:

deletemin: O(n) time linear search, replace key with X

decreasekey: O(1): write to array

RUNTIME OF DIJKSTRA'S

```
procedure dijkstra(G,ℓ,s)
for all u in V
        dist(u) := infinity
        prev(u) := nil
dist(s) := 0
H := makequeue(V)
while H is not empty
        u := deletemin(H)
        for all edges (u,v) in E
                if dist(v) > dist(u) + ℓ(u,v) then
                        dist(v):= dist(u) + ℓ(u,v)
                        prev(v):=u
                        decreasekey(H,v)
```

deletemin × |V| +
decreasekey × |E|

LOW-LEVEL DIJKSTRA

We've reduced the problem of creating a low-level efficient version of Dijkstra's algorithm to that of finding a good implementation of a priority queue

Total time will be

$$O(|V|(\text{deletemin}) + |E|(\text{decrease key}))$$

Different implementations of priority queue will have different trade-offs between costs of operations. Might not be a single one that is optimal for all kinds of graphs. So flexibility is useful

ARRAY AS A PRIORITY QUEUE

Array: indexed by vertex, giving key value.

deletemin: scan array for minimum

decreasekey: write new value

SAN	DEN	DFW	ORD	ATL	IAD		
0	inf.	inf	inf	inf	inf		
X	100	120	250	inf	inf		
X	X	120	200	inf	inf		
X	X	X	200	210	inf		
X	X	X	X	210	450		
X	X	X	X	X	270		

ARRAY AS A PRIORITY QUEUE

Array: indexed by vertex, giving key value.

deletemin: O(|V|)

decreasekey: O(1)

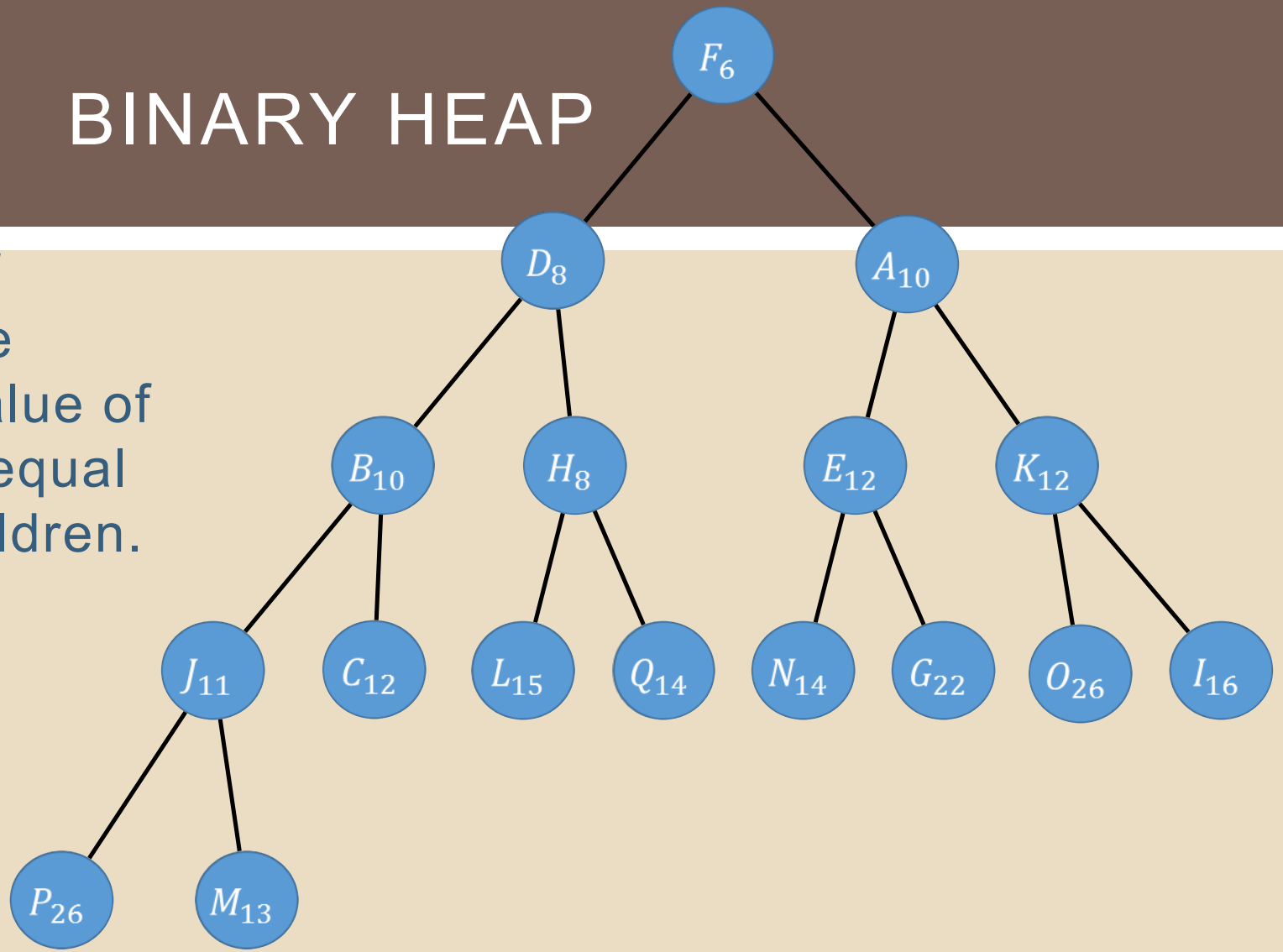
Total runtime:

|V|*deletemin + |E|*decreasekey

|V|*O(|V|) + |E|*O(1) = $O(|V|^2)$

BINARY HEAP

A complete binary tree of objects (vertices) with the property that each key value of an object is less than or equal to the key value of its children.



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

$$[F_6, D_8, A_{10}, B_{10}, H_8, E_{12}, K_{12}, J_{11}, C_{12}, L_{15}, Q_{14}, N_{14}, G_{22}, O_{26}, I_{16}, P_{26}, M_{13}]$$

BINARY HEAP

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

$[F_6, D_8, A_{10}, B_{10}, H_8, E_{12}, K_{12}, J_{11}, C_{12}, L_{15}, Q_{14}, N_{14}, G_{22}, O_{26}, I_{16}, P_{26}, M_{13}]$

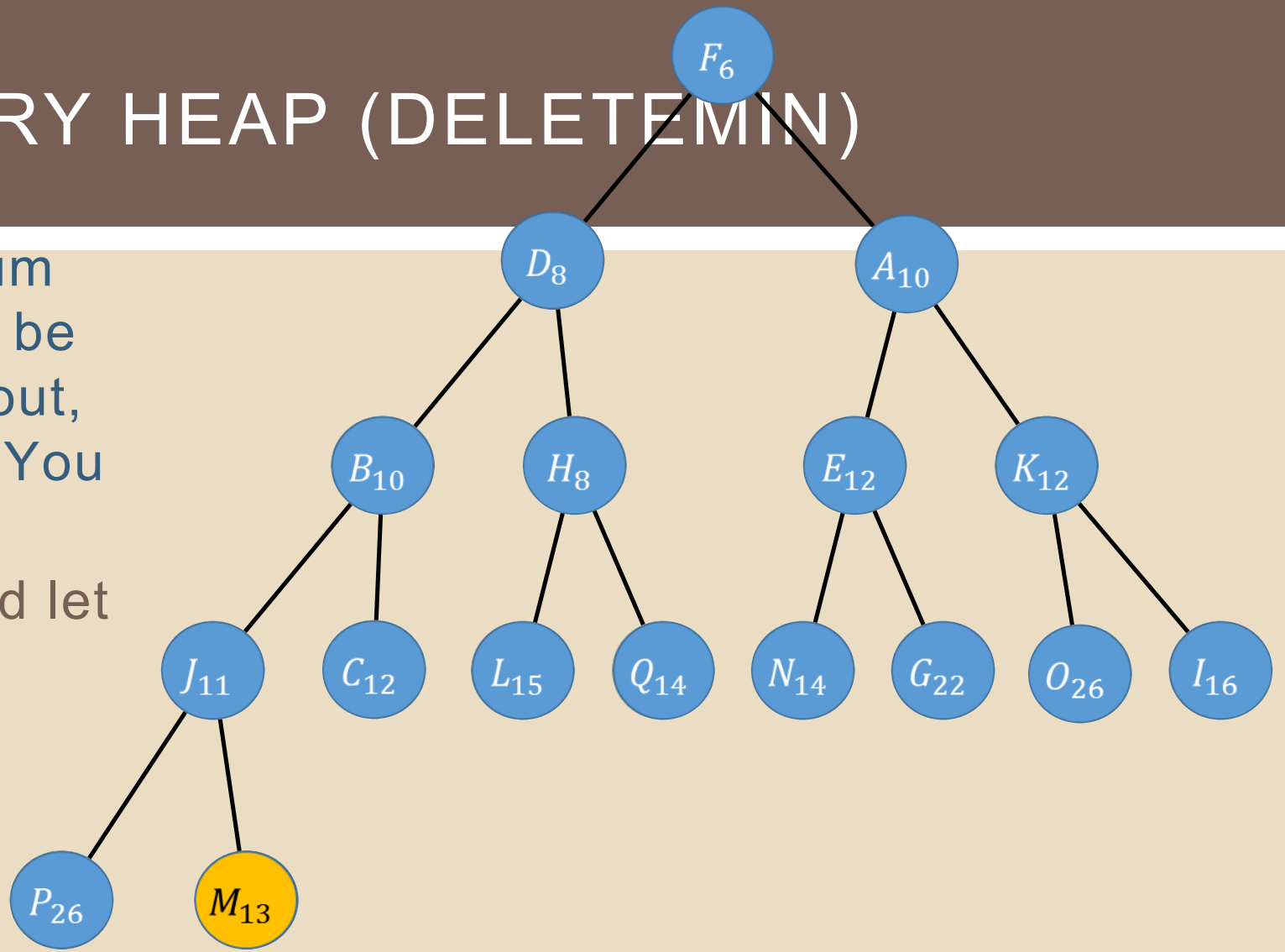
- The Binary heap can be implemented with an array a[n] of vertices.
- The children of a_i are a_{2i} and a_{2i+1}.
- The parent of a_i is $a_{\lfloor i/2 \rfloor}$.

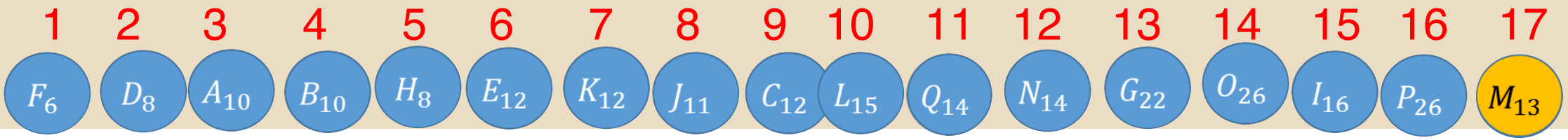
BINARY HEAP (DELETEMIN)

The object with the minimum key value is guaranteed to be the root. Once you take it out, you must reorder the tree. You replace the

root with the last object and let it trickle

down.



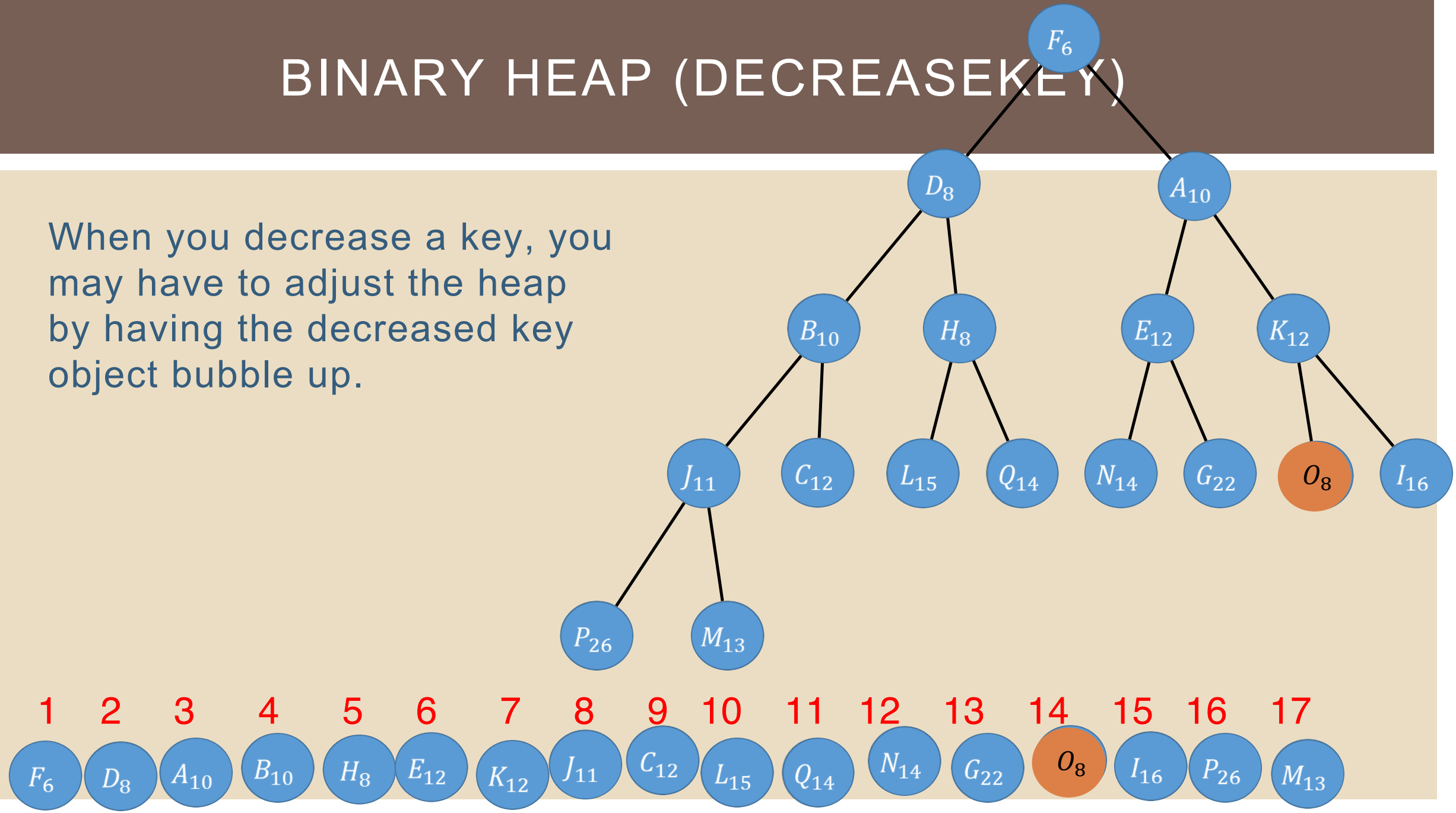


BINARY HEAP (DELETEMIN)

- When the last object is put in as the root, it may trickle down the entire length of the heap, the time taken is $O(\log(n))$ where n is the number of objects in the heap.
- When performing Dijkstra's algorithm, the number of objects in the heap is $|V|$ so the time taken is $O(\log(|V|))$.

BINARY HEAP (DECREASEKEY)

When you decrease a key, you may have to adjust the heap by having the decreased key object bubble up.



ONE COMPLICATION

How do we know where v is in the binary heap?

Keep a supplemental array (an address book) indexed by v, and keep pointers in both directions between this array and binary heap elements.

BINARY HEAP (DECREASEKEY)

- When the object decreases key, it may bubble up the entire heap, the time taken is $O(\log(n))$ where n is the number of objects in the heap.
- When performing Dijkstra's algorithm, the number of objects in the heap is $|V|$ so the time taken is $O(\log(|V|))$.

BINARY HEAP

deletemin: $O(\log(|V|))$

decreasekey: $O(\log(|V|))$

Dijkstra's takes time:

deletemin $\times|V|$ + decreasekey $\times|E|$

$$O(\log|V|)\times|V| + O(\log|V|)\times|E| = O\big((|V| + |E|)\log|V|\big)$$

ARRAY VS BINARY HEAP

	Array	Binary Heap
	$O(\|V\|^2)$	$O((\|V\| + \|E\|)\log(\|V\|))$
• Sparse graphs: $\|E\| = \Theta(\|V\|)$		
• Dense graphs: $\|E\| = \Theta(\|V\|^2)$		

B-ARY HEAP

What if we kept the heap idea, but made it larger fan-out?
Instead of one child, each vertex has B children.

When B is really large, almost like an array.

Depth: $\log_B n = logn / \log B$
Asymmetry: moving a vertex up the tree, only need to look at parents.
Moving a vertex down, we need to find the smallest child

B-ARY HEAP

Depth: $\log_B n = logn / \log B$

Asymmetry: moving a vertex up the tree, only need to look at parents
So reducing a key value only takes O(depth)= O(log n/log B).

Moving a vertex down, we need to find the smallest child.
So deleting the minimum element takes time O(B) at each level,
so O(B log n/log B) time.
Total time O((B/log B) n log n + m log n/ log B)).

B-ARY HEAP

Depth: $\log_B n = logn / \log B$

ReduceKey: O(log n/log B)

DeleteMin: O(B * logn/ log B)

Total Dijkstra time: O((B/log B) n log n + m log n/ log B)).

Optimization: pick B to set terms equal: Bn= m, B= m/n.

Total time O(m log n / log (m/n)). Note that when m is a power of n between 1 and 2, this is O(m).

ADVANTAGES OF FLEXIBILITY

- By working with a higher level version of the algorithm, we can understand what's going on without getting caught up in details
- Flexibility allows us to fit data structures to be efficient for the given circumstance

CORRECTNESS OF BW-DIJKSTRA

Let truebw(v) be the actual max bw from s to v.

We want to show that at the end of the algorithm bw(v) = truebw(v) for all vertices v.

In the middle of the algorithm, this is not necessarily true for all vertices. But once a vertex moves to X, its bw is locked in.

We will prove that dist(v) = truedist(v) for all v in X.

We will prove this by induction on k, the number of elements in X.

(Note: only one element is put into X at a time.)

CORRECTNESS OF BW-DIJKSTRA'S

Induction step: Assume for first k vertices to be finalized, bw(u)=truebw(u). Let v be the next vertex to be added to X, via an edge from u already in X.

The edge from u to v maximizes bw(v)= min (bw(u), w((u,v)))

Because there is a path reaching u with bw(u)=truebw(u) bandwidth,

There is a path of bandwidth bw(v) reaching v.

If there were a better path, it would have to leave X at some point,

from u' to v'. Then min (bw(u'),w((u',v')) $\leq bw(v)$ and the total bandwidth can only decrease from there. So the path has